

Open-Source Mapping Stack

ARCHITECTURE



QUICK START

```
cp .env.sample .env
python3 -m pip install -r requirements.txt
./start.sh
```

ENVIRONMENT (.ENV)

```
POSTGRES_DB=kenda_cases
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
DATABASE_URL=postgres://postgres:postgres\
@localhost:5432/kenda_cases
```

LIVE ENDPOINTS

Service	URL
Web map	http://localhost:8088
Martin TileJSON	http://localhost:3000/crimes
Vector tile	http://localhost:3000/crimes/{z}/{x}/{y}
Martin catalog	http://localhost:3000/catalog
PostGIS	localhost:5432 (psql / QGIS)

USEFUL COMMANDS

```
podman ps
psql $DATABASE_URL
psql $DATABASE_URL -c "SELECT count(*) FROM crimes;"
psql $DATABASE_URL -c "\d crimes"
python3 load_data.py
```

POSTGIS SCHEMA (KEY COLUMNS)

id	SERIAL PRIMARY KEY
title	TEXT NOT NULL
year	INTEGER
crime_type	TEXT
solved	BOOLEAN
lat/lon	DOUBLE PRECISION
geom	GEOMETRY(Point, 4326) — spatial index via GIST
-- Geometry is computed on insert: ST_SetSRID(ST_MakePoint(lon, lat), 4326)	

MARTIN CONFIG (MARTIN.YAML)

```
postgres:
  connection_string: ${DATABASE_URL}
  auto_publish:
    tables: true
  from_schemas: public
  listen_addresses: "0.0.0.0:3000"
```

MAPLIBRE GL JS (APP.JS)

```
Basemap OpenFreeMap dark style (no API key)
Source type: "vector" → Martin tile URL
Layers circle halo + filled dot, color-coded by solved
Sidebar Loaded from local crimes.json (filter / stats)
Popups On click: title, victim, year, weapon, description
```

FRONT-END CONFIG (TOP OF APP.JS)

```
const MARTIN_URL = "http://localhost:3000"; // tile server
const TABLE_ = "crimes"; // PostGIS table
const MAP_CENTER = [-104.8214, 38.8339]; // [lon, lat]
const MAP_ZOOM = 11;
const BASEMAP_STYLE =
  "https://tiles.openfreemap.org/styles/dark";
```

CONTAINER IMAGES

```
PostGIS docker.io/postgis/postgis:18-3.6
Martin ghcr.io/maplibre/martin:latest
Nginx standard nginx, serves /usr/share/nginx/html
```

Tip: Connect QGIS directly to PostGIS on localhost:5432 to inspect geometries, run spatial queries, or style layers independently of the web map.

1 · ROW COUNTS

```
SELECT
  count(*) AS total,
  count(*) FILTER (WHERE solved) AS solved,
  count(*) FILTER (WHERE NOT solved) AS unsolved
FROM crimes;
```

2 · ST_ASTEXT — SEE THE GEOMETRY

```
SELECT title, ST_AsText(geom) AS wkt
FROM crimes LIMIT 5;
```

Converts the binary geom column to human-readable WKT coordinates.

3 · ST_ASJSON — EXPORT A ROW

```
SELECT title, ST_AsJSON(geom) AS geojson
FROM crimes WHERE id = 1;
```

Same format Maplibre and most web APIs consume directly.

4 · ST_DWITHIN — RADIUS SEARCH

```
-- Crimes within 1 mile of downtown Colorado Springs
SELECT title, year, solved
FROM crimes
WHERE ST_DWithin(
  geom::geography,
  ST_MakePoint(-104.8214, 38.8339)::geography,
  1609 -- metres (1 mile)
)
ORDER BY year;
```

:geography cast → PostGIS works in metres; no projection math needed.

GOING FURTHER — POSTGIS

- `ST_ClusterKMeans` — cluster points for heatmaps
- `ST_Hexagon` — hexbin aggregation
- Raster support — elevation, imagery
- `pgbouncer` — connection pooling in production
- Book: *PostGIS In Action* (Manning Press)

GOING FURTHER — MARTIN & MAPLIBRE

- Martin: SQL functions as tile sources; MBTiles for offline; CDN caching
- Maplibre: heatmap layers; 3D terrain; Deck.gl integration; live GeoJSON sources

Jason Holden · Peak ETL · jason@peaketl.com

5 · KNN — NEAREST NEIGHBOURS

```
SELECT title,
  round(ST_Distance(
    geom::geography,
    ST_MakePoint(-104.8214, 38.8339)::geography
 )::numeric) AS dist_m
FROM crimes
ORDER BY geom::geography
<-> ST_MakePoint(-104.8214, 38.8339)::geography
LIMIT 5;
```

<-> is the KNN distance operator — index-accelerated via GIST.

6 · AGGREGATE BY DECADE

```
SELECT (year/10*10)::text||'s' AS decade,
  count(*) AS cases,
  round(100.0
    * count(*) FILTER (WHERE solved)
    / count(*) AS pct_solved
FROM crimes
GROUP BY decade ORDER BY decade;
```

7 · ST_BUFFER — ZONE OF INTEREST

```
WITH zone AS (
  SELECT ST_Buffer(
    ST_MakePoint(-104.8214, 38.8339)::geography,
    2000 -- metres (2 km radius)
  )::geometry AS circle
)
SELECT count(*) AS crimes_within_2km
FROM crimes, zone
WHERE ST_Within(geom, zone.circle);
```

8 · ST_CENTROID — GEOGRAPHIC CENTER

```
SELECT ST_AsText(
  ST_Centroid(ST_Collect(geom))
) AS center_of_all_crimes
FROM crimes;
```

ST_Collect aggregates all points; ST_Centroid finds their geographic center.

REFERENCES

- PostGIS postgis.net/documentation (<https://postgis.net/documentation/>)
- Martin maplibre.org/martin (<https://maplibre.org/martin/>)
- Maplibre GL JS maplibre.org/maplibre-gl-js/docs (<https://maplibre.org/maplibre-gl-js/docs/>)
- OpenFreeMap openfreemap.org (<https://openfreemap.org>) — free basemap, no API key
- QGIS qgis.org (<https://qgis.org>) — free desktop GIS
- Podman podman.io/docs (<https://podman.io/docs>)

GitHub: github.com/peaketl/open-source-mapping-presentation

Contact: jason@peaketl.com · (719) 310-5240 · www.peaketl.com